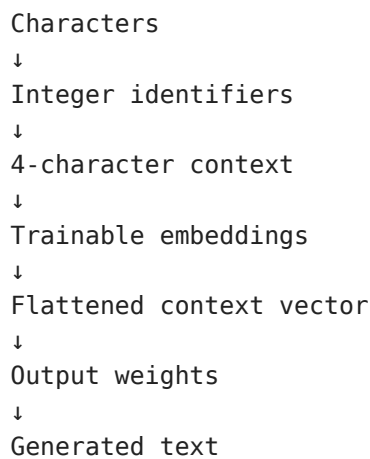


Report - Version 5: Embeddings and Context Window

Introduction

Version 5 extends the TensorFlow neural character language model from Version 4 with learned embeddings and a fixed context window.

The corpus, vocabulary, training and validation split, shuffled mini-batches, cross-entropy objective, automatic differentiation, and autoregressive sampling remain in place. The one-character one-hot input is replaced by four character identifiers whose learned embedding vectors are combined to predict the next character.



Goal of Version 5

The model should be able to:

- reuse the same 440-character corpus and 27-character vocabulary from Version 4;
- replace single-character one-hot inputs with fixed contexts of four character identifiers;
- map each character identifier to a trainable 8-dimensional embedding vector;
- concatenate four embeddings into a 32-value context representation;
- learn both the embedding matrix and output weights with `tf.GradientTape`;
- preserve mini-batch training, validation monitoring, and reproducible autoregressive generation;
- verify context construction, model shapes, gradients, parameter updates, deterministic retraining, and generation.

Architecture

The complete Version 5 flow is:

```
Corpus: 440 characters
↓
Vocabulary: 27 characters
↓
436 four-character context-target examples
↓
Integer input tensor: (436, 4)
↓
Embedding matrix: (27, 8)
↓
Flattened context: 32 values
↓
Output weight matrix: (32, 27)
↓
Reproducible 80 / 20 example split
↓
GradientTape mini-batch updates
↓
Sliding-window autoregressive generator
```

The context now contains four previous characters. Version 5 therefore moves beyond the one-character neural bigram used in Version 4 while keeping the model intentionally small and inspectable.

Context windows, split and mini-batches

Sliding windows create one input-target example for every position with four available previous characters. The same local NumPy generator with seed 42 shuffles the 436 example indices, and TensorFlow gather operations select the training and validation tensors.

```
training examples = 348
validation examples = 88
batch size = 32
updates per epoch = 11
```

Only the training examples are shuffled and used for parameter updates. Validation examples measure loss without modifying the model. Because neighboring context windows overlap, the example-level split keeps the same intentional small-dataset limitation as earlier versions.

Trainable embeddings and output weights

Each character identifier selects one row from a trainable embedding matrix. Four selected vectors are concatenated into one fixed-size representation, which is multiplied by a trainable output matrix to produce next-character logits.

```
Input tensor shape: (436, 4)
Embedding matrix shape: (27, 8)
Embedded context shape: (1, 4, 8)
Flattened context shape: (1, 32)
Output weight matrix shape: (32, 27)
Trainable parameters: 1080
```

The embedding matrix contains 216 values and the output matrix contains 864 values. Both are stored as TensorFlow variables and use float32 calculations.

Stable TensorFlow loss

The context identifiers are mapped to embeddings, reshaped into one context vector, and multiplied by the output weights. Sparse softmax cross-entropy measures the prediction error directly from the logits and integer target IDs.

```
embeddings = tf.gather(embedding_matrix, inputs)
context_vectors = tf.reshape(
    embeddings, (batch_size, 32)
)
logits = tf.matmul(context_vectors, output_weights)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

The initial train and validation losses are both about 3.2958, which is consistent with an almost uniform prediction over the 27-character vocabulary.

Controlled execution and reproducibility

The notebook keeps CPU execution and suppresses low-level TensorFlow logs because the model is small and does not require CUDA.

Explicit seeds control TensorFlow parameter initialization, NumPy data splitting, mini-batch shuffling, and text sampling. Repeated runs are deterministic within the same software environment.

Automatic differentiation

For each mini-batch, TensorFlow records the forward calculations and automatically derives the loss with respect to both trainable parameter sets:

- `tf.GradientTape` records the embedding lookup, reshape, matrix multiplication, and loss;
- `tape.gradient` calculates gradients for the embedding matrix and output weights;
- the sparse embedding gradient is converted to a dense tensor for this small vocabulary;
- `assign_sub` applies gradient descent updates to both variables;
- training and validation loss are measured after every epoch.

```
with tf.GradientTape() as tape:
    batch_loss = calculate_loss(...)
    embedding_gradient, output_gradient = tape.gradient(
        batch_loss, [embedding_matrix, output_weights]
    )
    embedding_matrix.assign_sub(learning_rate * embedding_gradient)
    output_weights.assign_sub(learning_rate * output_gradient)
```

Training configuration:

```
Learning rate: 1.0
Batch size: 32
Epochs: 100
Seed: 42
```

Training result

```
Initial train loss: 3.295823
Final train loss: 0.504322
Initial validation loss: 3.295813
Final validation loss: 6.635019
Best validation loss: 2.841331
Best validation epoch: 14
```

Training loss decreases strongly throughout the run. Validation loss initially decreases, reaches its minimum at epoch 14, and then increases substantially while training loss continues to improve.

This is clear overfitting: the larger context-based model learns the small training set very well but becomes less effective on validation examples. The notebook records the best epoch for analysis but uses the final epoch-100 parameters for generation.

Gradient and parameter checks

The tests verify that TensorFlow produces finite gradients for both trainable parameter sets, that the gradient shapes match the variables, and that both variables actually change during training.

```
embedding_gradient, output_gradient = tape.gradient(
    loss, [embedding_matrix, output_weights]
)
assert dense_embedding_gradient.shape == embedding_matrix.shape
assert output_gradient.shape == output_weights.shape
assert not np.allclose(initial_embeddings, trained_embeddings)
assert not np.allclose(initial_output_weights, trained_output_weights)
```

Learned probabilities

After training, the model produces a next-character probability distribution from a four-character context rather than from one character.

For the context mode, the highest probabilities are:

```
l: 0.8659   space: 0.0584   r: 0.0461
c: 0.0266   s: 0.0024      i: 0.0005
```

Neural text generation

TensorFlow calculates probabilities from the current four-character context, while a local NumPy generator samples the next character. The new character is appended, the oldest context character is removed, and the window moves forward by one position.

```
mode -> l
odel -> ?
...
```

Five samples from the trained context mode are all l with seed 42. The demonstration then generates 300 characters from the final parameters, and the same seed reproduces the same text.

Included checks

The notebook assertions verify context-window construction, tensor and variable types, parameter shapes, data splits, finite losses, normalized probabilities, finite gradients, parameter changes, deterministic retraining, reproducible generation, requested output length, and the starting context.

A complete top-to-bottom run ends with the message All checks passed.

What Version 5 teaches

The central progression is:

```
context -> IDs -> embeddings -> concatenate -> logits  
-> loss -> gradients -> update -> slide -> sample
```

Version 5 changes the predictive model for the first time since the neural bigram was introduced. The educational focus is now on learned token representations and using several previous characters as context.

Interpretation

Embeddings replace sparse one-hot vectors with compact trainable representations. A four-character context gives the model more local information than Version 4, and the separate positions remain distinguishable after the four embeddings are concatenated.

The model is still simple: the concatenated context is sent directly through a linear output projection. Its increased capacity fits the training corpus much more strongly, which also makes the observed overfitting easy to see.

Limitations

- the context has a fixed length of four characters;
 - the corpus is small and embedded directly in the notebook;
 - overlapping context windows can appear across the random training and validation split;
 - the flattened context has no recurrent state, attention mechanism, or hidden nonlinear layer;
 - there is no early stopping or checkpoint restoration, so generation uses epoch-100 parameters even though validation is best at epoch 14;
 - validation is not a robust estimate of generalization to a different corpus;
 - generated text captures stronger local character patterns but still has no long-term coherence.
-

Conclusion

Version 5 extends the TensorFlow character model with a fixed four-character context and learned embeddings. Character IDs select trainable vectors, the context vectors are concatenated, and both the embedding matrix and output weights are learned with automatic differentiation.

The fixed flattened context keeps the implementation easy to inspect and prepares the project for Version 6, where sequence modeling can process context as an ordered sequence.